Reliable Video Upload & Frame Integrity Detection

Confluentia 2026 · IIT (BHU) · Nihilent nSEPIA Technical Challenge · solo submission by Sreenandh S · November 2026
Code, evidence files and full documentation: the repository linked in the submission email. Every number here comes from a file in that repository.

A client sends a 30-second video to a backend while the network moves between 20% and 100% strength. The backend accepts the video only when every byte has arrived and the whole-file SHA-256 matches the client's. Only then does it decode the video, analyse it at 30 FPS and write an integrity report. The report gives the time and frame numbers of each missing, duplicated or damaged frame, and says whether the evidence points to transfer, capture or encoding, or that it cannot tell.

50 / 50	**0.933 / 0.995**	**5**	**82**
uploads COMPLETE with matching SHA-256 under 10 emulated network scenarios	detector recall / precision on 120 held-out known-fault videos	real browser runs, incl. webcam and iPhone Safari, with a per-frame capture log	automated tests (pytest), one command

1. How the system works

Client Python uploader, or the browser page (MediaRecorder + frame log) → **Upload API** FastAPI. Chunk + SHA-256 at a byte offset → **State machine** SQLite + fsync'd file. Verified prefix only → **Finalize gate** Re-hash whole file. Match → COMPLETE, else REJECTED → **Analysis** ffprobe → full decode → detect → 30 FPS timeline → capture log → **Report** JSON + HTML, tool versions, limitations

1. **Session.** The client declares the file size and SHA-256.
2. **Chunks.** Each chunk is sent to a byte offset with its own SHA-256. The server keeps only a contiguous, verified prefix and fsyncs it before recording it in SQLite, so the recorded offset is never ahead of the disk. A re-sent identical chunk changes nothing.
3. **One recovery path for every failure.** Dropped network, timeout, client crash or server crash: the client asks the server how many bytes it holds and continues from there. Retries use exponential backoff with full jitter (capped at 30 s); a connection with no progress for 20 s is abandoned and retried; the chunk size adapts between 64 KB and 8 MB (doubles after a success, halves after a failure).
4. **Finalize.** The server re-reads the file from disk and hashes it. Only a match gives `COMPLETE`. Server states: `UPLOADING`, `STALLED` (no completed chunk for a while), `FAILED`, `COMPLETE`, `REJECTED`; every change is logged with a reason.
5. **Analyse.** Analysis can open a file only through a function that refuses anything not `COMPLETE` with a matching hash.

2. Transfer under bad networks

Run on a separate Linux laptop (Ubuntu 24.04, kernel 7.0). netem shapes a private virtual link (rate, delay, loss in both directions); Toxiproxy in front of the server cuts, resets or hangs connections. Faults fire when the client's own log shows a given fraction of the file sent, so each fault lands mid-transfer. Test video 3,756,860 bytes, max chunk 512 KB. The link was measured before the runs: ping 22.2 ms at 100% and 401.7 ms at 20%.

STRENGTH	RATE	RTT	LOSS
100%	20 Mbit/s	20 ms	0%
60%	5 Mbit/s	80 ms	1%
20%	500 kbit/s	400 ms	5%

At 20% the link delivers only 140–200 kbit/s of useful data: with 5% loss and 400 ms delay TCP cannot fill 500 kbit/s. The 3.76 MB test video then takes 2.5–3.6 minutes, and it still completes.

SCENARIO (5 RUNS EACH)	WHAT HAPPENS	MEDIAN S	RETRIES	VERIFIED
stable_100 / 60 / 20	fixed strength	1.9 / 10.2 / 181	0	15/15
fluctuating	100 → 60 → 20 → 60 → 20 → 100 mid-transfer	32.2	0.4	5/5
interruption	at 30%, every connection cut and refused for 10 s	26.6	6.2	5/5
network_switch	100% link drops for 3 s, returns as a 20% link	66.0	4.6	5/5
resets	TCP resets for 2 s at 25%, 50%, 75%	17.0	8.8	5/5
timeouts	data silently dropped for 30 s, twice	70.2	5.8	5/5
client_restart	client killed (SIGKILL) at 40%, restarted	10.6	0	5/5
server_restart	server killed (SIGKILL) at 40%, restarted 3 s later	17.2	4.4	5/5

50/50 COMPLETE, server SHA-256 equal to client SHA-256 in every run. The raw logs were checked separately from the harness: in every run the acknowledged chunks plus the server's resume offsets cover exactly bytes 0 to 3,756,860. In 13 runs the client's offset jumps forward after a retry, each time by at most one chunk: the chunk had reached the server but its acknowledgement was lost, so the client continued from the server's offset instead of sending it again. After a server kill the client re-sent at most 524,288 bytes. Damaged chunks, incomplete uploads and a file changed during upload are never accepted (16 tests).

3. Frame integrity analysis

Validation and decoding are separate steps, and every error is recorded: ffprobe checks the container and stream; PyAV decodes every frame and records every decoder message with its packet time; the ffmpeg command line decodes the file a second time as an independent check. Detection runs on the file's own timestamps. Only after that is the 30 FPS timeline built (each slot mapped to a source frame), and duplicates created by that conversion are labelled as artefacts, not defects.

Anomaly types: missing frames, duplicated frames, possibly skipped frames, irregular timing, timestamp errors, corrupted frames and truncation. Each has a time range, frame indices, the evidence, a confidence and a likely cause.

Measured on known faults

A generator made 24 kinds of faulty 30 s video with known ground truth. The detector was tuned on seed 1, the code frozen, and seeds 2–6 (120 videos) scored once:

TYPE	EXPECTED	FOUND	RECALL	PRECISION
corrupted_frame	148	142	0.959	1.000
missing_frames	30	30	1.000	0.968
duplicated_frame	20	17	0.850	1.000
irregular_timing	10	8	0.800	1.000
possible_skipped_frames	10	6	0.600	1.000
truncation	5	5	1.000	1.000
All	**223**	**208**	**0.933**	**0.995**

Every video was classified correctly as valid or invalid, and there were no false alarms on clean control videos. All 15 misses and the one false positive are listed in the repository: byte flips that decode without any error (6), possibly skipped frames (4), duplicates in low-motion content (3) and timing jitter just under the threshold (2).

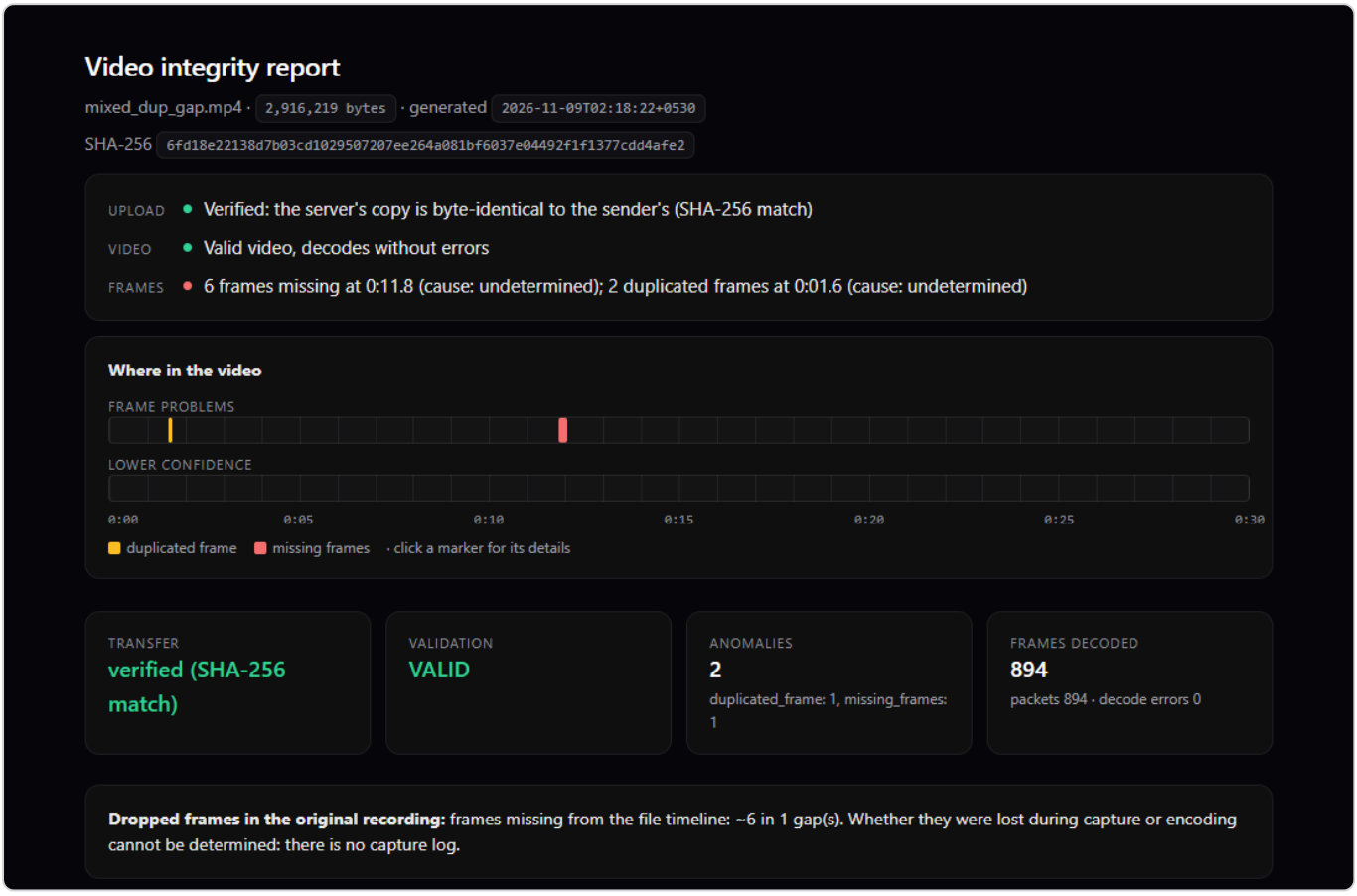
4. Transfer, capture or encoding?

A matching hash rules out transfer: the backend has exactly the bytes the client had. It says nothing about frames lost before that, in the camera or the encoder. Without more evidence the report says *cannot determine*, never *no drops*. The browser client adds that evidence: it logs every frame that reaches the page (`requestVideoFrameCallback`) and uploads the log with its own hash. A gap in the file that is also a gap in the log means the frames never arrived (**capture**); a gap only in the file means they arrived and were not recorded (**encoding**). The log's clock is aligned to the file's by a time search, and the result is used only if the frame pattern or a shared gap pins the offset down. Otherwise the report says the alignment is ambiguous.

BROWSER RUN	FINDING
Webcam, desktop Chrome	3.3 s gap at 0.30–3.60 s in file and log → capture. Two single frames shown in the page but missing from the file → encoding
iPhone, Safari	No delivery gaps; recorder started 3 ms after the request (Chrome: 1.5–3.8 s), so no start-up loss
Test pattern, 0.5 s stall injected	File gap 10.046–10.575 s, log gap 529.5 ms with no frames inside → capture
Test pattern / fake camera	Start-up loss located and attributed (encoding / capture)

5. Sample integrity report

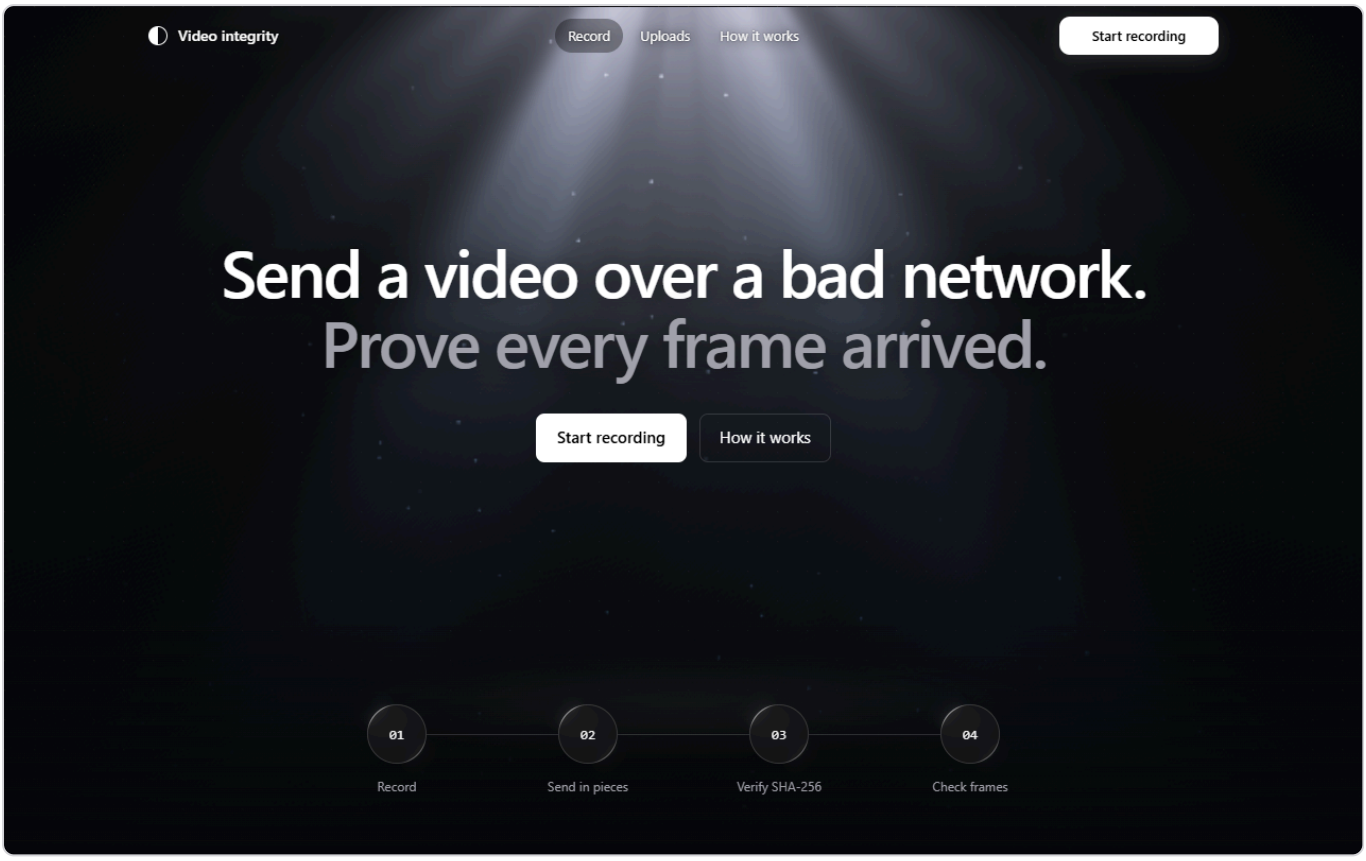
A 30 s video with frame 47 repeated twice and frames 356–361 removed (timestamps kept), uploaded through the system. Upload verified, video valid, both faults found at the right place. No capture log exists for this file, so the cause stays *undetermined*.



Top of the HTML report (docs/samples/mixed_dup_gap.html). The JSON next to it holds every frame and the tool versions (Python, OS, FFmpeg, PyAV, OpenCV, NumPy).

Two other samples: a file cut to about half its length (reported `VALID_WITH_ERRORS`, truncation from 15.33 s: "header declares 900 frames, file contains 462 packets"), and a text file named `.mp4` (`INVALID`, "Invalid data found when processing input").

6. The capture page



Record 30 s in the browser, watch it upload in verified pieces, read the verdict. The page's animations stop while recording and uploading so they do not compete with the frame log.

7. Limitations and what was not tested

- **A perfect transfer does not prove a clean recording.** Frames dropped before encoding with new timestamps leave no gap; only a motion jump hints at them (recall 0.60).
- **The capture log is written by the client.** The server checks its integrity, not its authenticity. Its time resolution is about 17 ms, and it is unreliable when the page is hidden or the browser is busy.
- **Detector limits:** low-motion duplicates and byte flips that decode without error are missed. Thresholds are fixed.
- **STALLED is a timer, not proof of a fault.** One slow first chunk on the 20% link was marked STALLED and then completed.
- **Network conditions were emulated.** The only real-device run is one iPhone recording over a hotspot. I did this project alone during end-semester exams, which left no time for a real mobile-network test or a real connection switch (in the emulated switch the client's address does not change). The test I would run next: the server on a second computer, both on one router, and the client laptop switching between the router's 2.4 GHz and 5 GHz networks halfway through an upload, so its address really changes. Pass: `COMPLETE`, hashes match, at most one chunk re-sent. A public tunnel was considered and rejected for this: through it the server would see every request coming from the tunnel process, so the address change would be invisible.
- Not tested: Firefox, the browser client under netem, resuming a browser upload after a page reload. No authentication; size limits only.

8. Run it

Python 3.12 or 3.13 and FFmpeg on PATH.

`pip install -r requirements.txt` · `python -m pytest -q` (82 tests, 4–6 min, run on mains power) · `uvicorn server.app:app --port 8000`, then open `http://127.0.0.1:8000/capture` · `python -m client.uploader video.mp4 --server http://127.0.0.1:8000`

Two WebGL effects on the capture page are ports of React Bits components (MIT + Commons Clause, licence file included). Repository map: `server/` upload API and pages, `client/` uploaders, `analysis/` validation, detection and reports, `nettest/` network harness and the 50-run evidence, `evidence/m5/` browser runs, `testset/` known-fault generator and scores, `docs/` design notes, limitations, demo script and samples.